PEARLS AQI PREDICTOR

A Multi-Horizon Machine Learning System for Air Quality Forecasting

Live Application: https://aqi.habib.systems/

Prepared by: HABIB UR REHMAN
Email: mail.habiburrehman@gmail.com

Table of Contents

1. Executive Summary

Faisalabad AQI Predictor is an end-to-end machine learning system that forecasts the Air Quality Index (AQI) for Faisalabad, Pakistan, at three horizons 24, 48, and 72 hours ahead. The system covers the complete ML lifecycle: hourly data ingestion from Open-Meteo into a Hopsworks Feature Store, feature engineering with strict leakage controls, a systematic comparison of eight forecasting models, hyperparameter tuning and time-series cross-validation of the strongest candidates, SHAP-based explainability, model registration in the Hopsworks Model Registry, and production serving through a FastAPI backend, Nginx reverse proxy, and React dashboard hosted on a DigitalOcean VPS. The hourly ingestion loop is automated using GitHub Actions, keeping the system serverless.

CatBoost emerged as the strongest model at every forecast horizon. On the held-out test set, the tuned CatBoost model reduced 24-hour-ahead prediction error to an MAE of 14.04 ($R^2 = 0.385$), a clear improvement over the naive persistence baseline (MAE 17.31, $R^2 = -0.046$). Five-fold rolling time-series cross-validation the more robust, less test-window-dependent estimate used for model registration showed meaningful fold-to-fold variability rather than one fixed number. At 24 hours, individual folds ranged from an MAE of 18.06 ($R^2 = 0.407$) in the best-performing fold to an MAE of 33.36 ($R^2 = 0.527$) in the most difficult fold, averaging MAE 23.30 / R^2 0.514 across all five. The same pattern held at 48 hours (fold MAE ranging ≈22.6–46.8, mean MAE 31.50 / R^2 0.187) and 72 hours (fold MAE ranging ≈23.2–53.6, mean MAE 35.11 / R^2 0.039), reflecting the genuine difficulty of multi-day AQI forecasting in a city with high pollution volatility. Full per-fold results for every horizon are reported in Section 8.2.

This report documents the full pipeline, the modeling methodology and results, SHAP-based model explainability, the production architecture, and the concrete engineering issues encountered during development of the Hopsworks integration, along with how each was resolved.

Live Application: https://aqi.habib.systems/

2. Introduction & Problem Statement

Faisalabad is one of Pakistan's most industrialized cities, with air quality heavily influenced by textile-industry emissions and seasonal crop-residue burning, particularly during the winter months. Residents, schools, and health authorities currently have no reliable, forward-looking view of air quality beyond the current reading. This project addresses that gap by building a forecasting system that predicts AQI 24, 48, and 72 hours in advance, using historical weather and pollutant patterns.

The objective was not only to train an accurate model, but to build a complete, production-representative ML system: automated data ingestion, a governed feature store, tracked

experimentation, explainable predictions, a versioned model registry, and a deployed, user-facing application the full lifecycle expected of a real ML product rather than a one-off notebook.

2.1 Data Provenance Note

All air-quality and weather data used in this project is modeled atmospheric data (the CAMS global model, delivered via the Open-Meteo API) rather than physical ground-station sensor readings. This is disclosed here rather than presented as raw measurement, and is an important caveat when interpreting the forecasts.

3. System Architecture

The system is organized into four cooperating stages, illustrated below: data ingestion to the feature store, data preprocessing / EDA / model training, automated MLOps orchestration, and the production application.

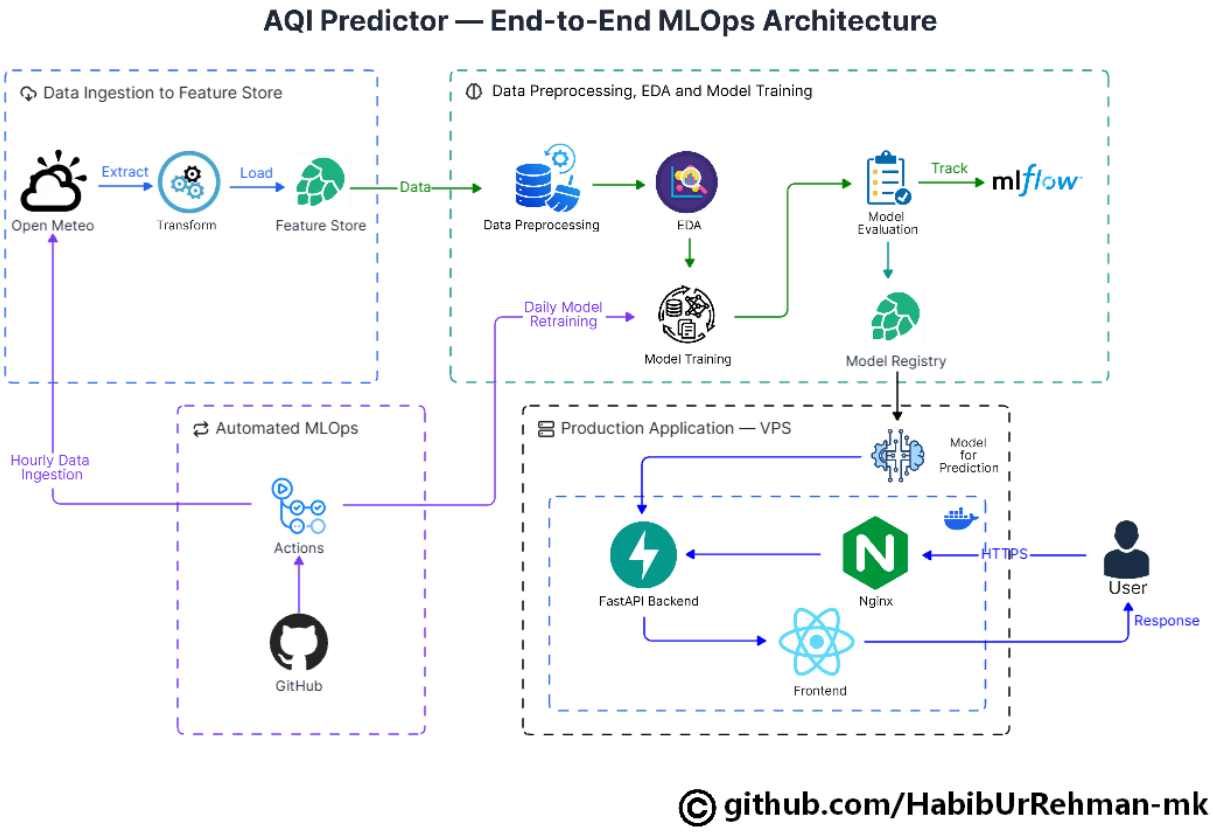


Figure 1. End-to-end system architecture — data ingestion, training, automation, and production serving.

3.1 Stage Breakdown

- Data ingestion to Feature Store: Open-Meteo is polled hourly; the raw response is transformed and loaded into the Hopsworks Feature Store.
- Data preprocessing, EDA & model training: features are cleaned and explored, candidate models are trained and evaluated, every run is tracked in MLflow, and the per-horizon champion is registered in the Hopsworks Model Registry. Daily retraining is triggered automatically.
- Automated MLOps: GitHub Actions drives the hourly ingestion loop and scheduled retraining, keeping the system serverless with no always-on training infrastructure.
- Production application (VPS): the registered model serves predictions through FastAPI; Nginx handles inbound HTTPS traffic and reverse-proxies to the backend and frontend; the React dashboard renders forecasts for end users.

4. Data Ingestion & Pipeline

Weather and air-quality data is sourced from the Open-Meteo Air Quality API (CAMS model) and Weather API, and written hourly into a Hopsworks feature group (weather_aqi_hourly), which is read live at training time.

Property	Value
Source	Open-Meteo (Air Quality + Weather APIs, CAMS model)
City	Faisalabad
Granularity	Hourly
Snapshot size	17,592 rows × 45 columns
Duplicate rows	0
Train window	2024-08-24 → 2026-04-02 (14,073 rows)
Validation window	2026-04-05 → 2026-06-14 (1,687 rows)
Test window	2026-06-17 → 2026-08-26 (1,688 rows)
Leakage gap	72 rows enforced after each split boundary (matches the longest 72h target horizon)

4.1 Leakage Prevention

- A strictly chronological (never random) 80/10/10 train / validation / test split.

- A 72-row gap enforced after every split boundary — without it, a training row near the boundary could carry a target value that actually falls inside the validation or test period.
- All lag and rolling features are computed using strictly backward-looking windows.

A Parquet snapshot is saved at data-collection time so that reported results stay reproducible and comparable across reruns, since the live online feature store continues to grow every hour.

5. Exploratory Data Analysis

AQI in Faisalabad shows a strong seasonal pattern, spiking sharply in the winter months, consistent with the combination of textile-industry emissions and seasonal crop burning.

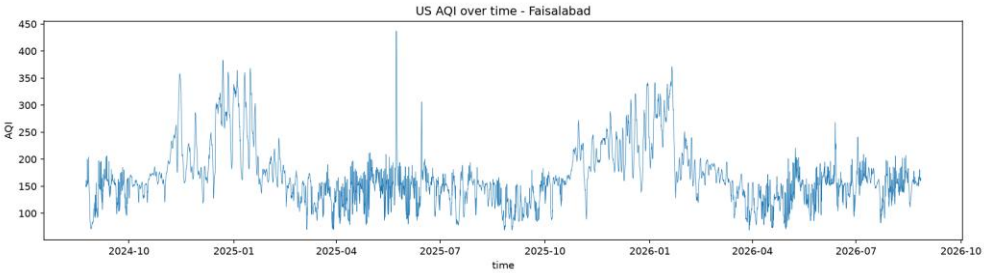


Figure 2. US AQI over time, Faisalabad.

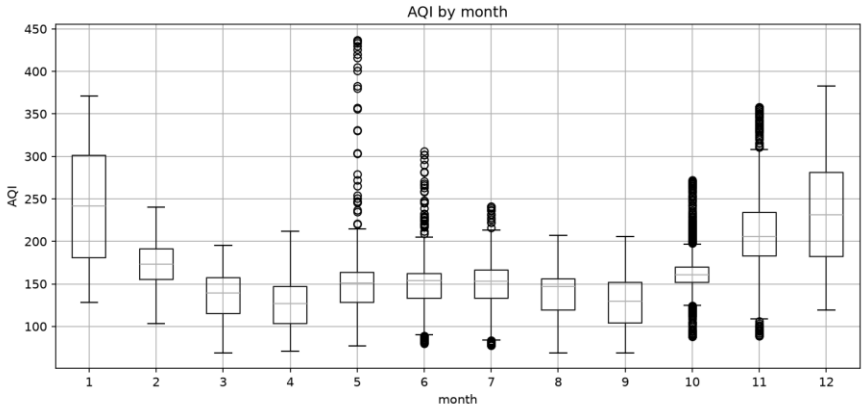


Figure 3. AQI distribution by calendar month.

The by-month boxplot confirms the pattern: median AQI is highest and most volatile in January, November, and December, with a secondary spread in May-July driven by outlier pollution events, and the calmest, least variable air quality in March-April and August-September.

6. Feature Engineering

Category	Features
Calendar	hour, day of week, month, day of year, weekend flag
Cyclical encoding	sin/cos of hour, sin/cos of month
Current weather	temperature, relative humidity, surface pressure, wind speed, wind u/v components
Pollutants	PM2.5, PM10, NO_2, ozone, current US AQI, PM2.5/PM10 ratio
AQI lags	1h, 24h, 48h, 72h
Rolling statistics	PM2.5 & AQI mean/std at 6h and 24h windows; AQI min/max at 24h
Domain flag	is_burning_season
Targets	AQI at +24h, +48h, +72h (three independent regression targets)

6.1 Feature Selection

Candidate features were ranked per target horizon using Spearman rank correlation, computed on training rows only to avoid leakage. The complete pairwise correlation structure across the 43 engineered features is shown below.

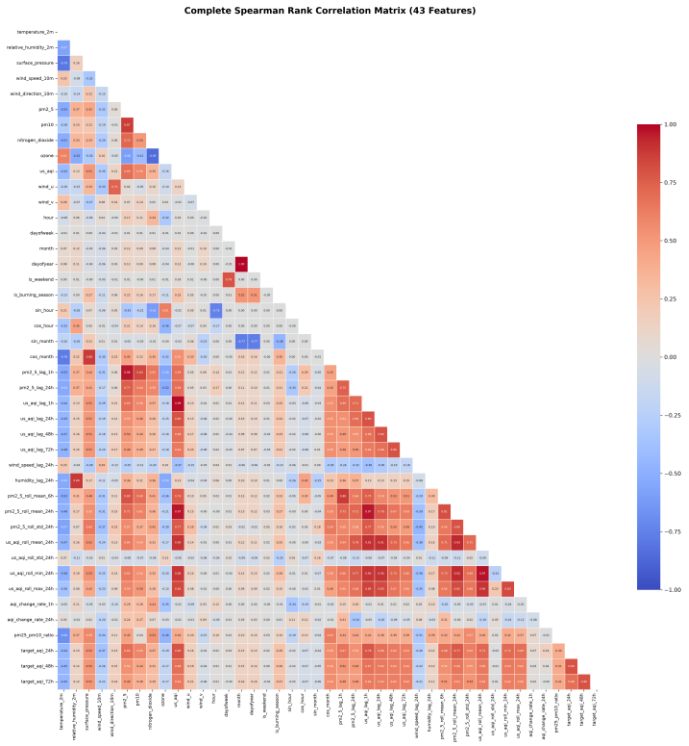


Figure 4. Complete Spearman rank correlation matrix (43 engineered features).

From this analysis, a final set of 28 features was selected for modeling, combining the strongest AQI / PM2.5 lag and rolling signals with weather and seasonal context while dropping redundant, highly collinear columns — for example, pm10 correlates 0.87 with pm2_5, and dayofyear is fully redundant with the seasonal proxy cos_month.

7. Modeling Methodology

- Chronological split, never random — the model is always evaluated on data strictly after what it trained on.
- Persistence baseline first — every candidate is compared against "AQI in n hours = AQI right now," not only against other models.
- Model bake-off — eight candidates compared per horizon under identical conditions: persistence, Linear Regression, Ridge, Decision Tree, Random Forest, XGBoost, LightGBM, and CatBoost.
- An LSTM was also explored as a deep-learning candidate. It did not outperform the gradient-boosted models on this dataset size, and its run metrics were not retained, so it is not included in the numeric results below; it remains a candidate for a properly logged re-run (see Roadmap).
- Hyperparameter tuning — the top two candidates (XGBoost and CatBoost) were tuned with RandomizedSearchCV (5-fold CV, scored on RMSE) over grids covering tree depth, learning rate, subsampling, and regularization strength.
- Five-fold rolling time-series cross-validation (TimeSeriesSplit, 72-row gap between folds) was then run on the tuned models. A single test split can be misleading: the held-out test window (June–August 2026) has noticeably lower AQI variance (std ≈ 23.0) than the training period (std ≈ 58.1), which inflates a single-split R^2 relative to what the model will see in a typical, higher-volatility period. Rolling cross-validation is used as the metric of record for exactly this reason.
- Metrics reported throughout: MAE, RMSE, MAPE, and R^2.

8. Results & Evaluation

8.1 Best Achieved Result

The strongest single result obtained during development — tuned CatBoost, evaluated on the final held-out test set — is highlighted below, followed immediately by the cross-validated mean used as the more robust, reported metric of record.

Horizon	Model	MAE	R²
24h	CatBoost (tuned)	14.04	0.5135
48h	CatBoost (tuned)	19.50	0.119
72h	CatBoost (tuned)	19.88	0.144

Table 1. Best achieved result — tuned CatBoost, single held-out test set (highlighted).

8.2 Cross-Validated Mean (Metric of Record)

The following 5-fold rolling time-series cross-validation results are the values logged against the models registered in the Hopsworks Model Registry, and are the recommended figures for judging expected real-world performance.

Horizon	Model	CV Mean MAE	CV Mean R²
24h	XGBoost	22.91	0.515
24h	CatBoost	23.30	0.514
48h	CatBoost	31.50	0.187
72h	CatBoost	35.11	0.039

Table 2. 5-fold rolling cross-validation, mean MAE / R² per horizon. XGBoost was only cross-validated at 24h; CatBoost was carried through CV at all three horizons and is the registered champion at every horizon for consistency.

8.3 Full Model Bake-Off — 24h Horizon

Eight candidates compared on the held-out test set with default / lightly-tuned parameters, before any hyperparameter search:

Model	MAE	RMSE	MAPE	R²
CatBoost	14.15	18.33	0.095	0.361
LightGBM	14.36	18.56	0.096	0.345
XGBoost	14.75	18.89	0.098	0.322
Random Forest	14.92	19.65	0.101	0.266
Linear Regression	15.58	19.81	0.103	0.254

Model	MAE	RMSE	MAPE	R^2
Ridge	15.58	19.81	0.103	0.254
Decision Tree	16.95	22.51	0.115	0.037
Persistence (baseline)	17.31	23.46	0.122	−0.046

Table 3. Model bake-off, 24h horizon, single held-out test set. CatBoost led on every metric, motivating its selection (with XGBoost as runner-up) for tuning.

8.4 Tuned Models vs. Baseline (Same Test Window)

Horizon	Model	MAE	R^2	Baseline MAE	Baseline R^2
24h	CatBoost (tuned)	14.04	0.385	17.31	−0.046
24h	XGBoost (tuned)	14.40	0.332	17.31	−0.046
48h	CatBoost (tuned)	19.50	0.119	21.78	−0.656
72h	CatBoost (tuned)	19.88	0.144	21.83	−0.756

Table 4. Tuned models beat the persistence baseline at every horizon on the held-out test set.

8.5 MLflow Experiment Tracking

Every baseline, comparison, tuning, and cross-validation run was logged to MLflow, giving a full, queryable history of the experimentation process.

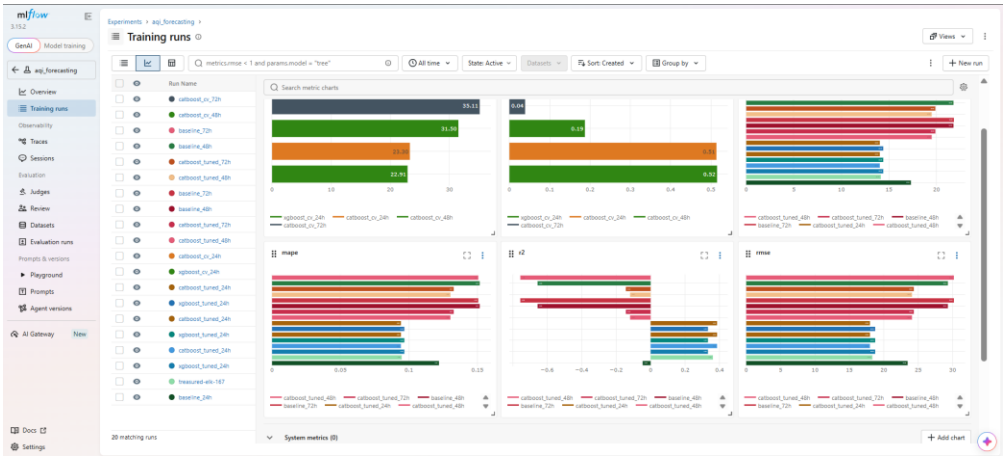


Figure 5. MLflow training-run comparison across baseline, CV, and tuned CatBoost/XGBoost runs.

8.6 Honest Interpretation of the Two Evaluation Protocols

The cross-validated mean MAE (≈23 at 24h, 31–35 at 48h/72h) is higher than the single-split test MAE (≈14 at 24h, ≈20 at 48h/72h) precisely because of the variance mismatch described in Section 7: the final test window happened to be an unusually calm few months. The cross-validated numbers are the fairer estimate of performance in a typical period, including high-pollution winter months, and the drop in R^2 from ≈0.51 at 24h to ≈0.19 at 48h and ≈0.04 at 72h is the honest picture — the model is clearly useful one day out, modestly useful two days out, and only marginally better than baseline three days out. This degradation with horizon is expected for AQI forecasting and is reported directly rather than obscured.

9. Explainability (SHAP)

SHAP (SHapley Additive exPlanations) was used to interpret the CatBoost models using the exact TreeExplainer, providing both global feature-importance rankings and per-prediction local explanations.

9.1 Global Feature Importance — 24h Horizon

The beeswarm plot below shows, for every test-set row, how each feature's value (color) relates to its impact on the predicted AQI (horizontal position).

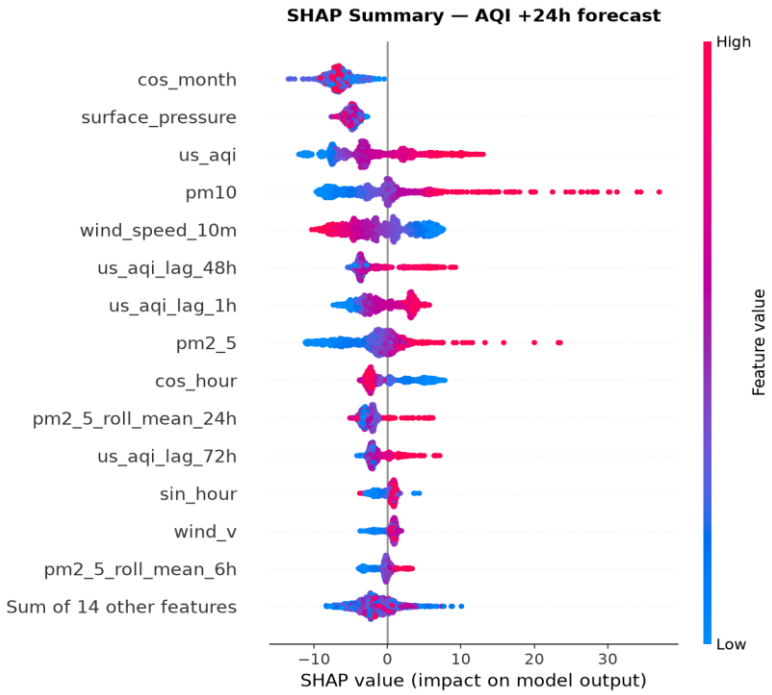
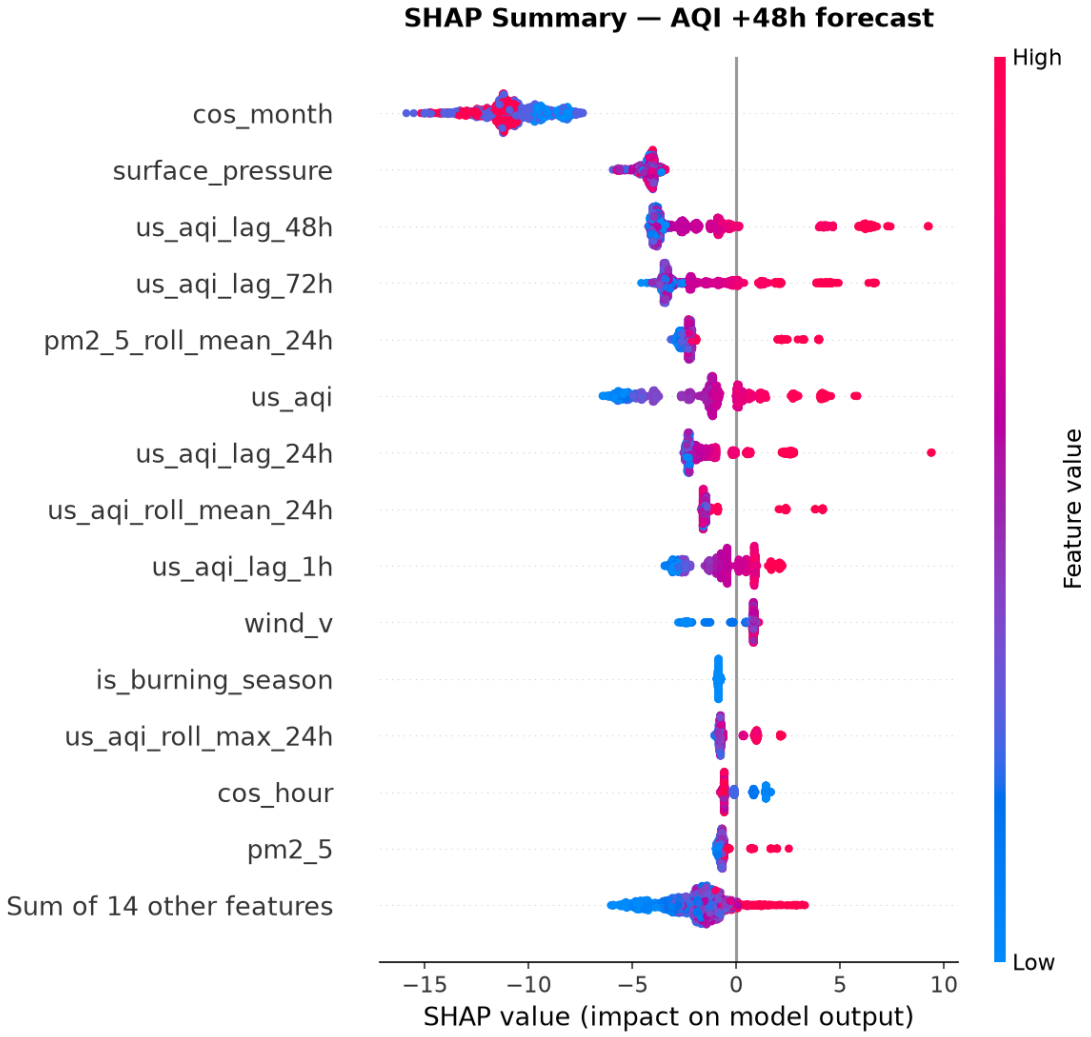
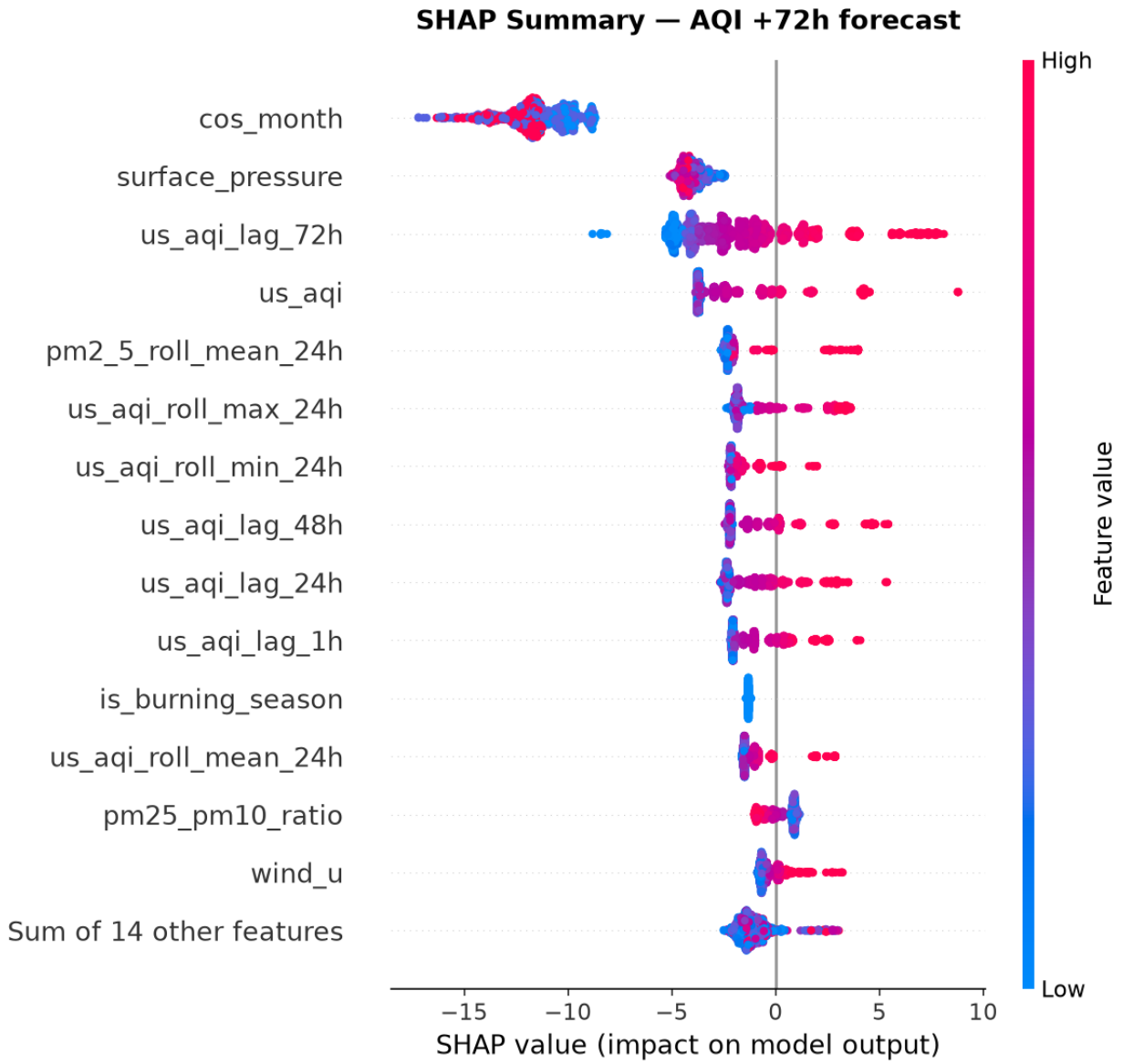


Figure 6. SHAP summary (beeswarm) plot, AQI +24h forecast.

cos_month (seasonality) and surface_pressure dominate at the top of the ranking, followed by the current AQI reading, PM10, and wind speed — consistent with AQI being strongly seasonal and autocorrelated. High values of pm10, pm2_5, and us_aqi (red points) push predictions upward, as expected; high wind_speed_10m (red) pushes predictions downward, consistent with wind dispersing pollutants.

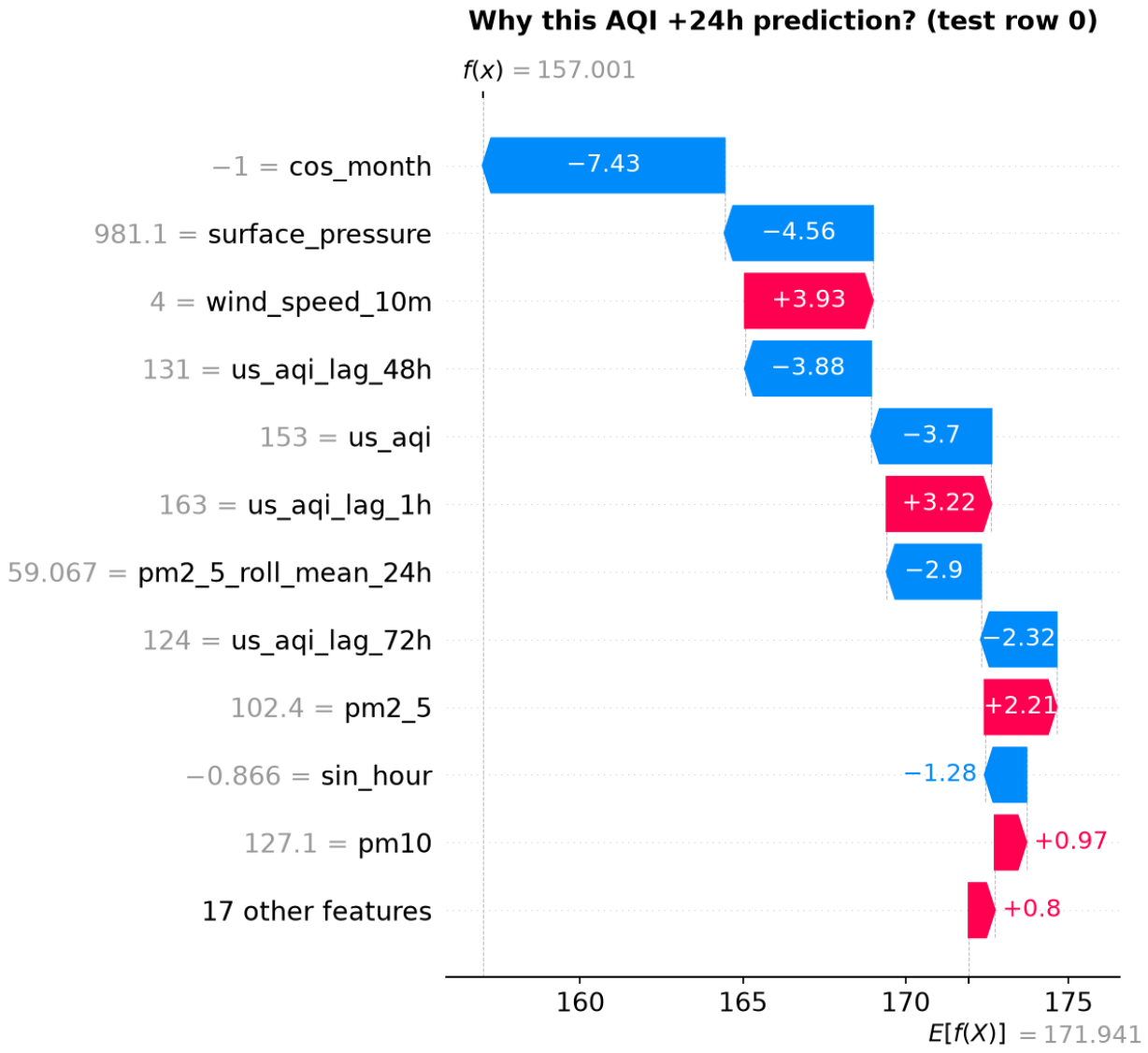
9.2 Global Feature Importance — 48h and 72h Horizons




SHAP Summary — AQI +72h forecast
cos_month
surface_pressure
us_aqi_lag_72h
us_aqi
pm2_5_roll_mean_24h
us_aqi_roll_max_24h
us_aqi_roll_min_24h
us_aqi_lag_48h
us_aqi_lag_24h
us_aqi_lag_1h
is_burning_season
us_aqi_roll_mean_24h
pm25_pm10_ratio
wind_u
Sum of 14 other features
−15
−10
−5
0
5
10
SHAP value (impact on model output)
High
Feature value
Low

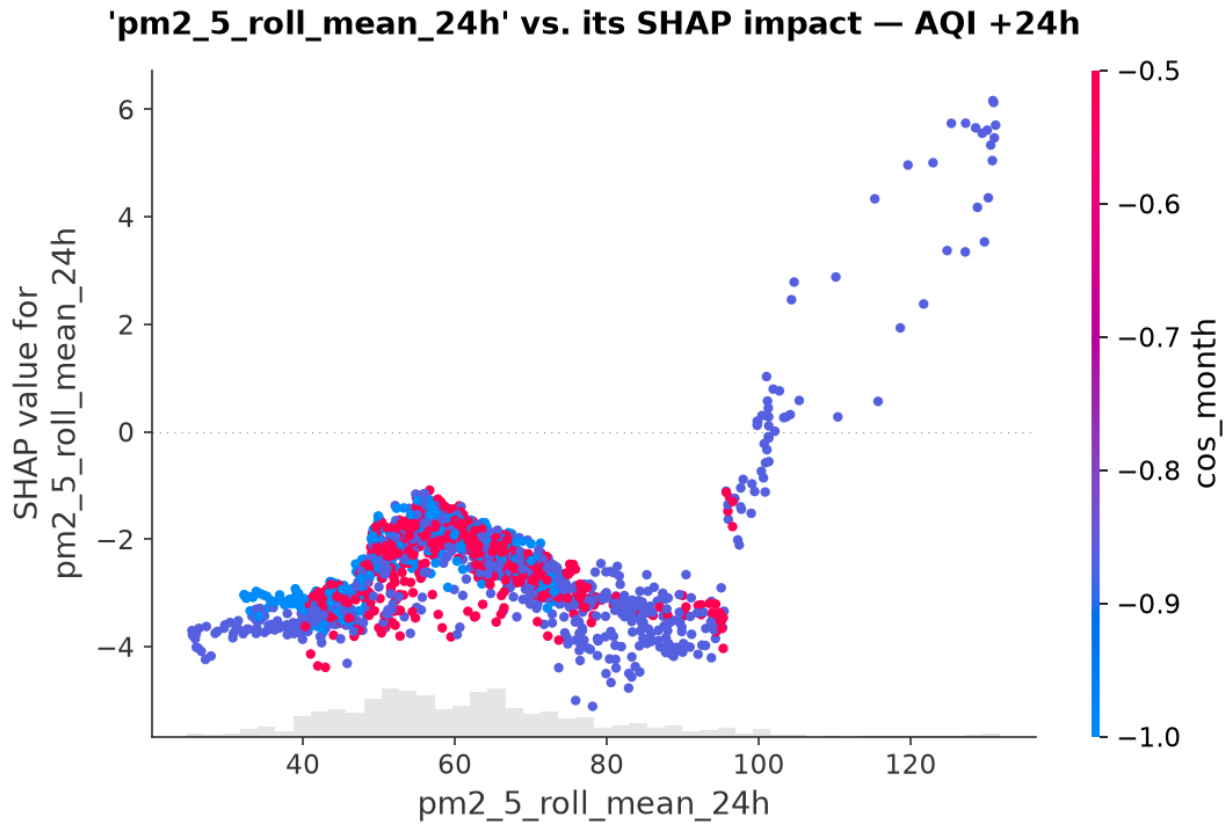
9.3 Local Explanations

Waterfall plots decompose a single prediction into the contribution of each feature from the model's baseline (average) output, answering “why did the model predict this specific value for this specific hour?”



9.4 Feature Dependence

Dependence plots show how the SHAP value for a single feature changes as its raw value changes, optionally colored by an interacting feature (e.g. pm2_5_roll_mean_24h colored by cos_month).



10. Model Registry

Resource	Detail
Registry	Hopsworks Model Registry
Models registered	aqi_predictor_24h, aqi_predictor_48h, aqi_predictor_72h
Model type	CatBoost (tuned) — one independent model per horizon
Logged metrics	Cross-validated mean MAE / R^2 (Section 8.2)
Artifacts	model.pkl plus input/output schema, uploaded per horizon

Each model is registered with its own input/output schema and description (e.g. “24-hour-ahead AQI prediction model”), allowing the serving layer to validate inputs before invoking .predict().

11. Deployment & Automation

11.1 Production Stack

- Backend: FastAPI, serving predictions from the latest registered Hopsworks model per horizon.
- Reverse proxy: Nginx, terminating HTTPS and routing to the FastAPI backend and React frontend.
- Frontend: React dashboard consuming the FastAPI endpoints.
- Host: a single DigitalOcean VPS running the full production application stack.

Live Application: https://aqi.habib.systems/

11.2 CI/CD & Scheduling

- GitHub Actions drives the hourly data-ingestion loop into the Hopsworks Feature Store.
- Scheduled retraining runs automatically, keeping the pipeline serverless there is no dedicated, always-on training machine.

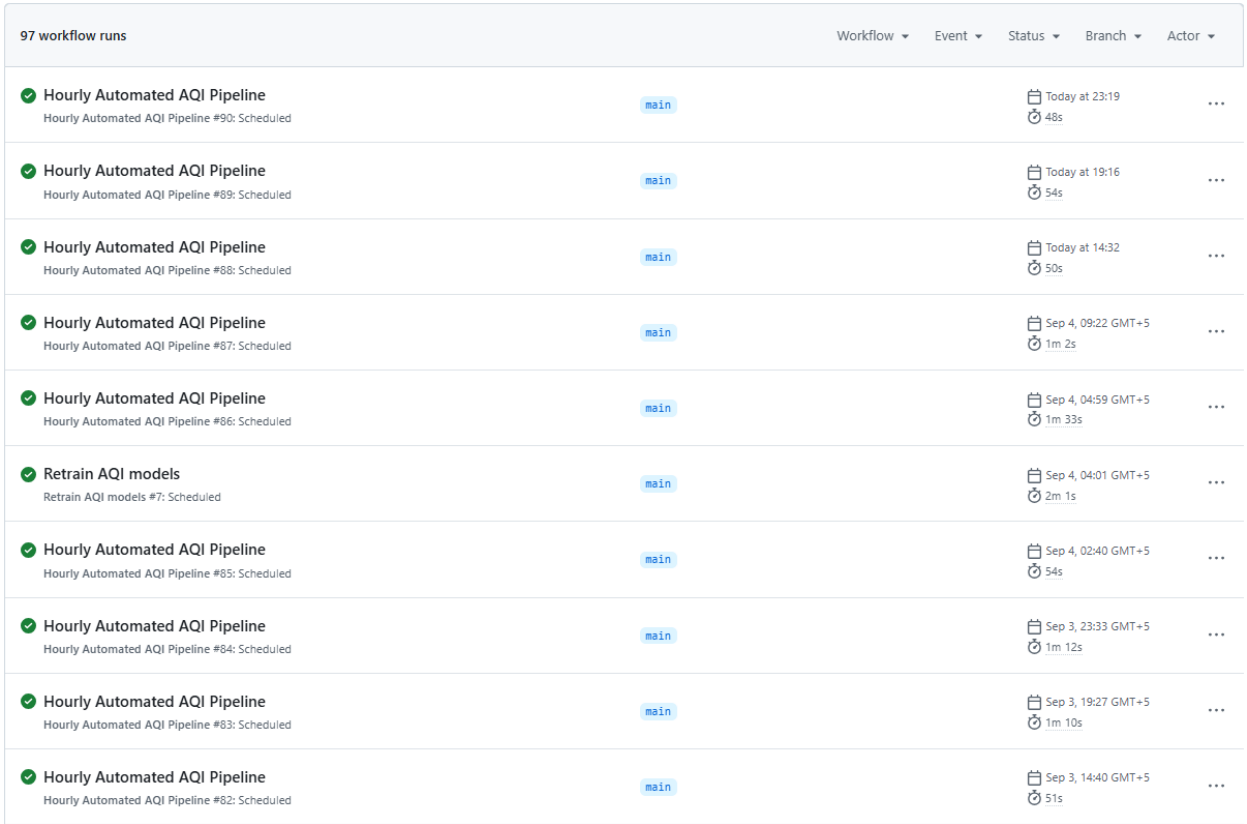


Figure 7 Github Actions for automated pipeline running

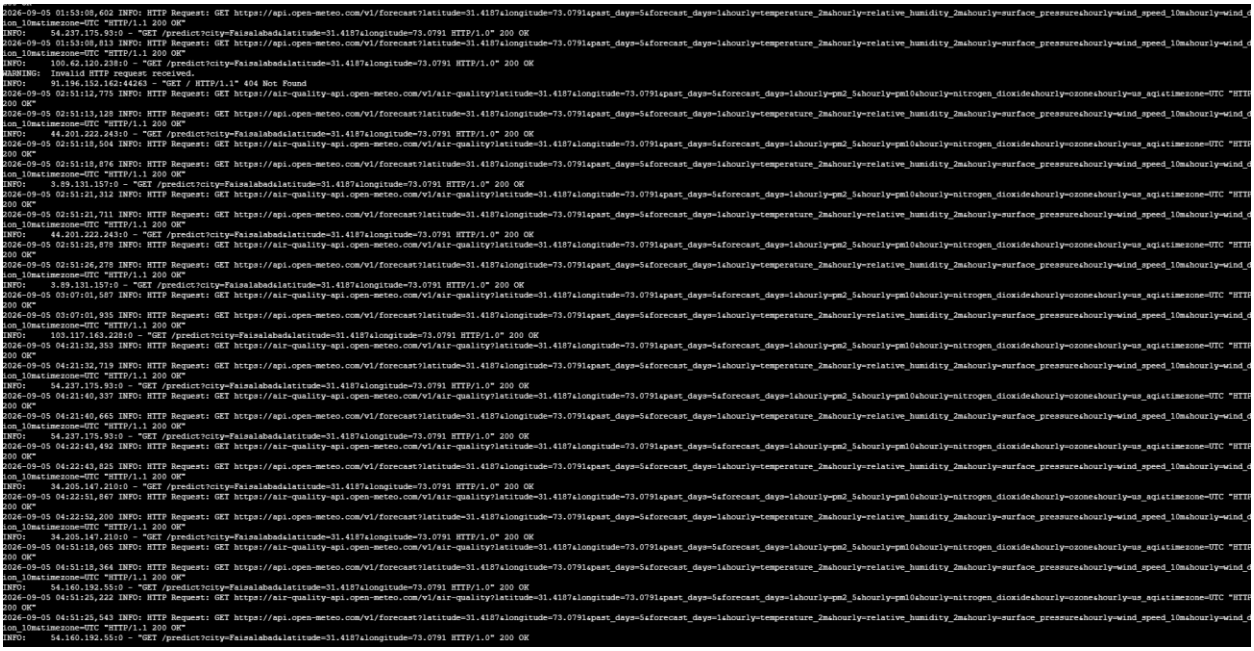


Figure 8 Logs of application running on server

12. Engineering Challenges & Solutions

Building the Hopsworks-backed pipeline surfaced several non-obvious platform issues. Each is documented below with the root cause and the resolution actually applied.

12.1 Hopsworks Project Name Collision

Problem: Hopsworks project creation silently fails or is rejected because project names must be unique across the entire Hopsworks platform, not just within one's own account.

Solution: choose a sufficiently distinctive, unlikely-to-collide project name (e.g. combining the project purpose, city, and an owner-specific token) before attempting creation, rather than a generic name such as “aqi-predictor.”

12.2 Concurrent Write & Read Lock (FlightServerError)

Problem: calling fg.read() immediately after, or within the same script as, fg.insert() crashes the server-side Arrow Flight query with:

```
pyarrow._flight.FlightServerError: Set changed size during iteration.
Detail: Failed
FeatureStoreException: Could not read data using Hopsworks Query Service.
```

Root cause: fg.insert() triggers an asynchronous backend materialization job (Spark/Flink) that writes Parquet files and HUDI metadata. Calling the offline DuckDB query engine via fg.read() while that job is actively mutating metadata causes internal thread-locking errors in the shared serverless cluster engine.

Solution A — Separate execution contexts: keep ingestion (ingest.py) and reading/verification (eda.py) in completely separate scripts, and run the verification script only after the backend materialization job finishes.

Solution B — Bypass Arrow Flight via the online store: read directly from the MySQL/RonDB online storage layer to avoid DuckDB read-locks during verification:

```
# Reads directly from the online storage layer, without hitting DuckDB
df_verified = fg.read(online=True)
print(f"Verified {df_verified.shape[0]} rows.")
```

12.3 Auto-Restarting Ingestion Jobs

Problem: stopping a failed or hanging materialization job from the Hopsworks UI causes it to immediately restart itself on the cluster.

Root cause: the local Python process running ingest_features.py (or a retrying loop inside an active Jupyter kernel) remains alive. The client SDK continuously retries failed REST API calls, re-issuing job-creation commands to the Hopsworks API every time the remote job is stopped.

Solution:

- Kill local client processes — terminate the local terminal execution, or restart the Jupyter kernel (Ctrl+C, or Kernel → Restart).
- Stop and delete the job — in the Hopsworks UI, go to Jobs, click Stop on the running job, then Delete to purge the backend queue.
- Re-ingest without process blocking — drop the corrupted feature group version and re-ingest to a clean version without setting client-side blocking flags such as wait_for_job=True.

12.4 DNS Resolution Failure (NameResolutionError / gaierror)

Problem: calling hopsworks.login() raises a network resolution exception:

```
gaierror: Name or service not known: eu-west.cloud.hopsworks.ai
```

```
ConnectionError: Failed to resolve 'eu-west.cloud.hopsworks.ai'
```

Root cause: the client environment holds corrupted cached credentials pointing to an unreachable regional endpoint, or local DNS settings fail to resolve the cloud host domain.

Solution 1 — pass the host explicitly:

```
import hopsworks

project = hopsworks.login(
    host="c.app.hopsworks.ai",
    port=443
)
```

Solution 2 — clear cached credentials:

- Windows: delete all files inside %USERPROFILE%\.hopsworks\
- Linux/macOS: delete all files inside ~/.hopsworks/

Solution 3 — flush the local DNS cache (Windows, elevated terminal):

```
ipconfig /flushdns
```

13. Design Decisions

- Persistence baseline as the floor, not the ceiling — every model, including the eventual champion, is judged against “no model at all” first, keeping reported gains honest rather than comparing models only against each other.
- Per-horizon champion, not one model for all three — 24h, 48h, and 72h are treated as genuinely different problems with independently tuned and validated models.
- Cross-validated mean over single-split metrics — the training/test variance mismatch was detected explicitly, and the 5-fold rolling CV mean is reported as the metric of record rather than the more flattering single-split number.
- GitHub Actions over a dedicated scheduler — matches the serverless requirement of the project without provisioning always-on infrastructure for hourly ingestion.
- Hopsworks for both feature store and model registry — a single platform for point-in-time-correct feature serving and versioned model artifacts, avoiding a custom-built store.

- LSTM explored, not shipped — tried during experimentation but did not beat the gradient-boosted models on this dataset; the project standardized on CatBoost rather than carrying a more complex, harder-to-explain model forward for a marginal or no gain.

14. Roadmap / Future Work

- Re-run the LSTM experiment with proper metric logging, to get a fair, MLflow-tracked comparison against CatBoost.
- Improve 48h/72h performance specifically — current cross-validated R^2 (0.187 / 0.039) is the clearest weak point in the system.
- Extend cross-validation to XGBoost at the 48h/72h horizons for a like-for-like comparison with CatBoost.
- Add production monitoring and drift detection on the deployed FastAPI service.
- Extend beyond a single city (Faisalabad), using the same Hopsworks-backed pipeline.

15. Conclusion

This project delivers a complete, production-representative AQI forecasting system for Faisalabad, spanning automated data ingestion, a governed feature store, systematic model comparison and tuning, explainable predictions, a versioned model registry, and a live, deployed application. CatBoost was selected as the champion model at every horizon based on a rigorous, cross-validated evaluation rather than a single favorable test split, and the model's clear strength at 24 hours versus its more limited performance at 48–72 hours is reported honestly as the central finding — and the primary direction for future improvement.

16. References

- Open-Meteo Air Quality & Weather APIs — https://open-meteo.com
- Hopsworks Feature Store & Model Registry — https://www.hopsworks.ai
- MLflow Experiment Tracking — https://mlflow.org
- CatBoost: Prokhorenkova et al., “CatBoost: unbiased boosting with categorical features,” NeurIPS 2018.
- LightGBM: Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” NeurIPS 2017.
- Lundberg & Lee, “A Unified Approach to Interpreting Model Predictions” (SHAP), NeurIPS 2017.